

Python Tables Study Guide

A practical companion for tabular data with pandas

This guide shows how to read, transform, combine, validate, and export tables with Python. Work through the accompanying `Python_Tables_Course.ipynb` for runnable examples and a final project. The supplied files describe 20 teams in the 2015–16 English Premier League season. A *row* is one team, and a *column* is one measured variable. Allow about two hours for the notebook and another 20 minutes for review.

Before you begin

You need basic Python arithmetic and indexing, Python 3, pandas, matplotlib, and Jupyter. Keep the notebook and the three CSV files in the same folder. From that folder, start Jupyter with `jupyter lab` or `jupyter notebook`. If a package is missing, run `python -m pip install pandas matplotlib jupyter` in your chosen Python environment. Run notebook cells in order; the examples build on variables created earlier.

File	Purpose
<code>EPLresults.csv</code>	Home and away wins, draws, losses, goals for, and goals against.
<code>EPLteams.csv</code>	Payroll, manager, manager hire date, and nationality.
<code>EPLFinalStandings.csv</code>	A reference result for checking your calculated standings.

1 The table model

A pandas `DataFrame` is a two-dimensional table with labeled columns and an index for rows. Each column has a data type, such as integer, floating-point number, text, date/time, or category. A single selected column is usually a `Series`; selecting a list of columns produces another `DataFrame`. Labels describe meaning, so choose column names that remain clear when the table is printed or exported.

```
# Import the file and inspect its shape, names, types, and missing values.
import pandas as pd
results = pd.read_csv("EPLresults.csv")
print(results.shape)          # (20, 11)
print(results.columns)
print(results.dtypes)
print(results.isna().sum())
print(results.head())
```

Check types and missing values before arithmetic. CSV is a text format and does not store Python data types explicitly. pandas infers them when reading.

2 Select, filter, and sort

Use `loc` for row labels and column names, and `iloc` for integer positions. A Boolean expression filters rows. Use parentheses around each comparison when combining conditions with `&` or `|`. Sorting reorders entire rows, keeping team names aligned with statistics.

```
# Keep team names with both win columns.
wins = results.loc[:, ["Team", "Home Wins", "Away Wins"]]

# Use zero-based positions for the first three rows.
first_three = wins.iloc[:3]

# Find teams with at least ten away wins.
strong_away = results.loc[results["Away Wins"] >= 10]

# Break a home-win tie with away wins.
ordered = results.sort_values(
    ["Home Wins", "Away Wins"], ascending=[False, False]
)
```

Common mistake. `df["Team"]` is a `Series`, while `df[["Team"]]` is a one-column `DataFrame`. Check which form a later operation expects. `iloc[:3]` includes rows at positions 0, 1, and 2; the stop position is excluded.

3 Calculate a standings table

Add corresponding home and away statistics for each team. Points equal three times total wins plus total draws; goal difference equals goals for minus goals against. The rows in these files are aligned within `EPLresults.csv`, so elementwise addition is appropriate. Rebuild a separate table to keep the imported data intact.

```
# Create total records for all five season statistics.
standings = pd.DataFrame({"Team": results["Team"]})
for stat in ("Wins", "Draws", "Losses", "GF", "GA"):
    standings[f"Total{stat}"] = (
        results[f"Home {stat}"] + results[f"Away {stat}"]
    )

# Apply the point rule and order by the stated tie breaker.
standings["GoalDiff"] = standings["TotalGF"] - standings["TotalGA"]
standings["Points"] = (
    3 * standings["TotalWins"] + standings["TotalDraws"]
)
standings = standings.sort_values(
    ["Points", "GoalDiff"], ascending=[False, False]
).reset_index(drop=True)
```

The five pairs must be added by their shared statistic names. Avoid blindly adding all numeric columns if their order or meaning might differ. The season has 38 matches per team; this is a useful arithmetic check. The supplied standings confirm all 20 rows and columns for this dataset. Other competitions can require additional tie breakers.

```
# Verify the match count and compare against the reference file.
assert (
    standings["TotalWins"]
    + standings["TotalDraws"]
    + standings["TotalLosses"] == 38
).all()
reference = pd.read_csv("EPLFinalStandings.csv")
pd.testing.assert_frame_equal(standings, reference, check_dtype=False)
```

4 Join related tables

A join combines rows with a common key. Here, `Team` identifies a team in each CSV. Check key uniqueness before a one-to-one merge. A left join keeps every standings row, and the merge indicator reveals missing matches. Unmatched fields contain missing values.

```
# Match each standings row to one team-information row.
teams = pd.read_csv("EPLteams.csv")
assert standings["Team"].is_unique
assert teams["Team"].is_unique
joined = standings.merge(
    teams, on="Team", how="left",
    validate="one_to_one", indicator=True
)
assert joined["_merge"].eq("both").all()
joined = joined.drop(columns="_merge")
```

A duplicate key can multiply rows in a merge. A misspelled team name can leave unmatched data. Both errors can be harder to notice after additional calculations, so validate immediately.

5 Manage names, types, and dates

Column names may include spaces or units. Rename them when a concise name improves later code, but keep units visible in the replacement. Dates in `EPLteams.csv` use day, abbreviated month name, and four-digit year; an explicit format avoids ambiguity. `errors="raise"` stops at an invalid date instead of silently producing missing data.

```
# Give frequently used fields concise names with units retained.
analysis = joined.rename(columns={
    "Payroll (M£)": "Payroll_MGBP",
    "Manager Hire Date": "ManagerHireDate",
    "Manager Nationality": "ManagerNationality",
})

# Parse dates and represent repeated labels as categories.
analysis["ManagerHireDate"] = pd.to_datetime(
    analysis["ManagerHireDate"], format="%d %b %Y", errors="raise"
)
analysis["ManagerNationality"] = (
    analysis["ManagerNationality"].astype("category")
)
```

Use `df.loc[:, columns]` to reorder or retain columns, `rename(columns={...})` to change names, and `drop(columns=...)` to remove columns. Most pandas methods return a new table unless assigned back; this makes it easier to inspect each transformation.

6 Export and verify

Write a CSV without the pandas index, then read it back. This checks that the intended columns and values survived serialization. CSV does not preserve category or date types, so parse these again when

loading a saved analysis table.

```
# Save and reopen the calculated standings.
standings.to_csv("PythonFinalStandings.csv", index=False)
reloaded = pd.read_csv("PythonFinalStandings.csv")
pd.testing.assert_frame_equal(reloaded, standings, check_dtype=False)
```

7 Explore without overclaiming

A scatter plot can reveal patterns or unusual teams. The horizontal axis below is payroll in millions of pounds; the vertical axis is league points. Correlation summarizes linear association but does not identify a cause. The dataset contains only 20 teams from one season. Club resources, transfers, injuries, and other factors may affect both quantities.

```
# Make a labeled descriptive plot of payroll and points.
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(7, 4))
ax.scatter(analysis["Payroll_MGBP"], analysis["Points"])
ax.set(xlabel="Payroll (million GBP)", ylabel="League points")
fig.tight_layout()
plt.show()
```

Recorded manager hire dates describe the manager named in the team file. Some dates occur after the season started on 8 August 2015. A tenure measure anchored to that date would then be negative, so inspect dates and define the research question before using tenure to explain season results.

8 Practice and answers

1. **Selection.** Show Team and Away GF for teams with at least ten away wins, highest away goals first.
2. **Tie breaker.** Which two teams earned 66 points, and why does one rank higher?
3. **Join check.** What happens to the team-information columns in a left join if a team key has no match?
4. **Project.** Produce a five-column report with team, points, goal difference, payroll, and manager nationality. Verify 20 unique teams and no missing values. Save it as `PythonTeamReport.csv`.

Answers and checks. For item 1, use the following selection and sort:

```
# Keep the requested rows and columns, then order by away goals.
answer = results.loc[results["Away Wins"] >= 10,
                    ["Team", "Away GF"]]
answer = answer.sort_values("Away GF", ascending=False)
```

For item 2, Manchester City and Manchester United both have 66 points; their goal differences are 30 and 14, respectively. For item 3, unmatched fields are missing, and an indicator column shows `left_only`. A reference solution for item 4 is in the final notebook cell. Describe a plot as an observed association; a causal conclusion needs a different study design.

Quick reference

Task	pandas expression
Read or write	<code>pd.read_csv("file.csv") / df.to_csv("file.csv", index=False)</code>
Inspect	<code>df.shape, df.dtypes, df.isna().sum(), df.describe()</code>
Select	<code>df.loc[:, ["Team", "Points"]] / df.iloc[:5]</code>
Filter	<code>df.loc[df["Points"] >= 60]</code>
Sort	<code>df.sort_values("Points", ascending=False)</code>
Add a column	<code>df["GoalDiff"] = df["TotalGF"] - df["TotalGA"]</code>
Join	<code>left.merge(right, on="Team", validate="one_to_one")</code>
Parse dates	<code>pd.to_datetime(df["date"], format="%d %b %Y")</code>